



SAPIENZA
UNIVERSITÀ DI ROMA

COMPLEMENTARY TRAINING ACTIVITY
Times Series Forecasting

Professor: Enrico Tronci

Student: Mojtaba Taj

Student ID: 1993339

Academic Year: 2024–2025

Contents

1	Long Short-Term Memory	2
1.1	Forget Gate	2
1.2	Input Gate	3
1.3	Output Gate	4
1.4	LSTM for Times Series Forecasting	4
2	Methodology	5
2.1	Data Preprocessing and Cleaning	5
2.2	Data Visualization	5
2.3	LSTM Model Architecture and Training	7
2.4	Model Performance Evaluation	8
2.4.1	Mean Absolute Error (MAE)	8
2.4.2	Mean Squared Error (MSE)	9
2.4.3	Root Mean Squared Error (RMSE)	9
2.4.4	R ² Score	9
2.5	Visualization of Predictions and Performance	10
2.6	T_Prediction	10
2.7	First Evaluation	12
2.7.1	Finding_T	14
2.8	Second Evaluation	15
3	Conclusion	16

1 Long Short-Term Memory

LSTM, or Long Short-Term Memory, is a type of artificial neural network architecture specifically designed for processing and predicting sequential data. It is a special variant of recurrent neural networks (RNNs) that overcomes the limitations of traditional RNNs, particularly the problem of vanishing and exploding gradients during backpropagation.

At its core, an LSTM network is designed to retain information over long periods, making it highly effective for tasks involving sequences, such as time-series forecasting, speech recognition, natural language processing, and video analysis. Traditional RNNs struggle with long-term dependencies because their gradients tend to diminish as they propagate backward through time, making it difficult for the network to learn from distant past inputs. LSTMs address this issue by introducing a memory cell and gating mechanisms that regulate the flow of information.

The key components of an LSTM cell include the forget gate, input gate, and output gate. The forget gate determines how much of the previous memory should be discarded. The input gate regulates how much new information should be added to the memory. The output gate decides what part of the memory should be used as the current output. These gates work together to ensure that relevant information is preserved while unnecessary data is discarded, allowing the network to maintain long-term dependencies effectively.

Mathematically, an LSTM cell involves a series of equations that update its internal state and produce an output at each time step. The memory cell, or cell state, acts as a conveyor belt that carries information across time steps with minimal modifications unless explicitly altered by the gates. The gating mechanisms use sigmoid and tanh activation functions to control the flow of information and ensure smooth learning.

LSTMs have been widely adopted in various fields due to their ability to process sequential patterns. They are commonly used in machine translation, text generation, speech synthesis, and even financial modeling, where long-term dependencies play a crucial role. Despite their advantages, LSTMs can be computationally expensive due to their complex structure and numerous parameters, which has led to the development of more efficient alternatives like Gated Recurrent Units (GRUs) and Transformer models.

1.1 Forget Gate

The forget gate determines how much of the previous memory should be discarded. This is crucial because not all past information is useful for future predictions. The forget gate uses a sigmoid activation function (which outputs values between 0 and 1) to decide whether each piece of information should be kept (values close to 1) or forgotten (values close to 0).

Mathematically, the forget gate is computed as:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (1)$$

where:

- W_f and b_f are the weight matrix and bias for the forget gate.
- h_{t-1} is the hidden state from the previous time step.
- x_t is the current input.
- σ is the sigmoid activation function.

After computing f_t , the forget gate multiplies it with the previous memory C_{t-1} to decide how much should be retained.

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t \quad (2)$$

where i_t and $\tilde{C}_t$ come from the input gate (explained next).

1.2 Input Gate

The input gate controls how much new information should be added to the memory. It consists of two parts:

- A sigmoid function (i_t) that determines which values to update.
- A tanh function ($\tilde{C}_t$) that creates a candidate memory update.

The formulas are:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (3)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (4)$$

The sigmoid function i_t decides how much of the candidate memory $\tilde{C}_t$ should be added to the current memory state C_t .

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t \quad (5)$$

This ensures that only relevant information from the new input is added to the memory.

1.3 Output Gate

The output gate decides what part of the memory should be sent as the output at the current time step. It also uses a sigmoid activation function to filter the information and a tanh function to scale it.

The equations for the output gate are:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (6)$$

$$h_t = o_t \cdot \tanh(C_t) \quad (7)$$

Here:

- o_t determines which parts of the memory state should be output.
- The memory state C_t is processed through a tanh function (which scales values between -1 and 1) before being multiplied by o_t to get the hidden state h_t .
- This hidden state h_t is then passed to the next time step and can also be used as the final output of the LSTM.

1.4 LSTM for Times Series Forecasting

Traditional time series forecasting models such as ARIMA, exponential smoothing, and regression-based approaches depend on assumptions regarding linearity and stationarity, making them less effective when handling complex, non-linear dependencies. In contrast, LSTMs learn patterns from data automatically without needing predefined mathematical relationships, which makes them highly effective for real-world forecasting tasks.

Unlike standard recurrent neural networks (RNNs), which struggle to retain long-term dependencies due to the vanishing gradient problem, LSTMs effectively maintain information over extended sequences. This capability makes them particularly useful when past observations—sometimes from many time steps ago—are crucial for accurate predictions. For example, in financial markets, economic cycles or seasonal trends spanning months or years can significantly influence short-term fluctuations.

Additionally, LSTMs are more flexible than traditional models because they do not require evenly spaced time intervals. This is particularly valuable for applications like energy demand forecasting or sensor-based anomaly detection, where missing or irregular data points are common. Their ability to handle multi-step forecasting (predicting multiple future values) further enhances their applicability, making them a powerful tool in domains such as weather prediction, sales forecasting, and healthcare analytics.

By learning directly from raw data without requiring manual feature engineering, LSTMs offer a robust solution for capturing both short-term fluctuations and long-term trends, outperforming many conventional forecasting techniques in complex, data-driven environments.

2 Methodology

The primary objective is to process, analyze, and model time-dependent sensor readings, predict future values, and evaluate forecasting accuracy. The methodology follows a structured approach, including data preprocessing, visualization, model training, evaluation, and prediction analysis.

2.1 Data Preprocessing and Cleaning

The dataset consists of three primary columns: timestamp, sensor ID, and recorded value. Each entry represents a sensor reading at a given time. The overall number of sensors is more than 50,000, and each sensor ID has more than 5,000 values. Given the nature of sensor data, preprocessing is critical to ensure data integrity. This includes handling missing values, removing outliers, normalizing data, and aligning timestamps to ensure consistency across different sensors.

The preprocessing steps include:

- **Handling missing values:** Null entries are filtered out to maintain data consistency.
- **Standardizing timestamps:** Timestamps for each sensor are reset relative to their first recorded observation, ensuring uniform tracking across multiple sensors.
- **Feature scaling:** MinMaxScaler is applied to normalize sensor values between 0 and 1, which enhances LSTM performance by preventing scale-related issues.
- **Sliding window transformation:** The dataset is restructured into supervised learning format, where past observations (X) are used to predict future values (Y). The time window is defined by the input parameter, which sets the number of past time steps considered for prediction. The future_step parameter determines how many future values need to be averaged for the prediction. This enables the model to forecast a mean value over the defined future period, rather than predicting individual future values. However, it can be adjusted based on different approaches. For this scenario, I am considering only the next value for the future time steps.

2.2 Data Visualization

A visualization function (`plot_sensors_data`) is implemented to graphically represent sensor readings over time. The visualization provides insights into trends and fluctuations. This step is essential for identifying seasonality, trend patterns, and irregularities within the data. The function ensures clear and informative plots with appropriate figure sizes and labeling. For each sensor, null values are ignored, meaning sensors with missing data will have fewer recorded values than those without missing data.



Figure 1: Sensors Reading

2.3 LSTM Model Architecture and Training

The forecasting model is based on an LSTM neural network due to its ability to capture long-term dependencies in sequential data. A separate LSTM model is trained for each unique sensor ID. The architecture consists of:

- **Two LSTM layers:** The first layer has 50 units with return sequences enabled, while the second layer has 25 units with return sequences disabled.
- **Dropout layers:** Applied to prevent overfitting by randomly deactivating a fraction of neurons during training.
- **Dense (fully connected) layers:** These refine time-dependent features extracted by the LSTM layers.
- **Mean Squared Error (MSE) loss function:** Used for measuring prediction accuracy.
- **Adam optimizer:** Selected for its efficiency in training deep learning models.
- **Early stopping mechanism:** Monitors validation loss and halts training if no improvement is observed after five consecutive epochs.

To ensure a fair model evaluation, the dataset is split into:

- Training set (50%) – Used for model learning.
- Test set (50%) – Used for performance assessment.
- Validation set (10%) (subset of training data) – Used for hyperparameter tuning.

Algorithm 1 LSTM-based Time Series Prediction for Individual Sensors

Input: Time series data for each sensor, sequence length, training/validation split ratio

Output: Trained LSTM model for each sensor

for each sensor **do**

 Create LSTM model architecture:

- LSTM Layer 1
- Dropout
- LSTM Layer 2
- Dropout
- Dense Layer 1
- Dense Layer 2

 Compile model with Adam optimizer and MSE loss function

 Configure early stopping

 Split training data (90% training, 10% validation)

 Train model

 Make predictions and evaluate results

end for

2.4 Model Performance Evaluation

Once trained, the model is evaluated on unseen test data. Predictions are rescaled back to their original scale using the MinMaxScaler. The performance of the model is assessed using several error metrics:

2.4.1 Mean Absolute Error (MAE)

The Mean Absolute Error (MAE) measures the average magnitude of errors in a set of predictions without considering their direction. It represents the average absolute difference between the predicted values and the actual observations. MAE is given by:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (8)$$

where y_i are the actual value, $\hat{y}_i$ are the predicted value, and n is the total number of observations. MAE treats all errors with equal weight and is less sensitive to outliers compared to squared error metrics like MSE and RMSE. It is expressed in the same units as the original data, making it directly interpretable. Lower MAE values indicate better model performance, with a value of 0 representing perfect predictions. MAE is particularly useful when the distribution of errors is not expected to be normal or when outliers should not have a disproportionate influence on the evaluation metric.

2.4.2 Mean Squared Error (MSE)

The Mean Squared Error (MSE) calculates the average squared differences between the actual and predicted values. It penalizes larger errors more significantly than MAE due to the squaring operation, making it particularly useful when large errors are especially undesirable.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (9)$$

where y_i are the observed values, $\hat{y}_i$ are the predicted values, and n is the number of observations. MSE is always non-negative, with values closer to zero indicating better model performance. Unlike MAE, MSE produces values in squared units of the original data, which can make direct interpretation less intuitive. This is one reason why RMSE (the square root of MSE) is often preferred when reporting final model performance metrics. MSE is differentiable everywhere, making it a suitable loss function for optimization algorithms in machine learning, particularly for regression problems.

2.4.3 Root Mean Squared Error (RMSE)

The Root Mean Squared Error (RMSE) is the square root of the MSE, providing a metric in the same unit as the original data. This makes RMSE directly interpretable in the context of the data being analyzed. RMSE gives higher weight to larger errors due to the squaring operation, making it particularly sensitive to outliers. Mathematically, it is defined as:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (10)$$

where y_i are the observed values, $\hat{y}_i$ are the predicted values, and n is the number of observations. Lower RMSE values indicate better model performance, with an RMSE of 0 representing a perfect fit to the data.

2.4.4 R² Score

The R² score, also known as the coefficient of determination, measures the proportion of variance in the dependent variable that is predictable from the independent variables. It quantifies how well the model explains the observed data, with values ranging from 0 to 1 (or negative in case of poor fits). An R² of 1 indicates that the model perfectly predicts the dependent variable, while an R² of 0 indicates that the model provides no predictive information.

Mathematically, it is defined as:

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} \quad (11)$$

where y_i are the observed values, $\hat{y}_i$ are the predicted values, and $\bar{y}$ is the mean of the observed values.

Error metrics are computed for each sensor, allowing a detailed performance analysis.

2.5 Visualization of Predictions and Performance

To further analyze model performance, the following visualizations are generated:

- **Training loss curve:** Displays the loss reduction over epochs, providing insights into model convergence.
- **Predicted vs. actual values plot:** Compares LSTM-generated predictions with real sensor values to assess forecasting accuracy.
- **Error metrics bar chart:** Summarizes MAE, MSE, RMSE, and R^2 Score for each sensor, facilitating comparative analysis.

Each visualization is saved in high resolution for further reference, ensuring clear and interpretable results.

2.6 T_Prediction

A specialized analysis function (`predict_T`) is incorporated to estimate the time required for a sensor reading to increase by a specified percentage. This feature is useful for:

- **Predictive maintenance:** Estimating when a machine sensor value will exceed a critical limit.
- **Trend forecasting:** Determining when environmental sensor values will cross a predefined threshold.

The function identifies the smallest time step (T) at which the predicted value surpasses a given threshold.

$$x(t + T) \geq (1 + p) \cdot x(t) \quad (12)$$

If no such increase is detected within the forecast horizon, the function returns -1, indicating that the increase does not occur within the specified period.

Algorithm 2 LSTM-Based Time Series Forecasting

```
1: Start
2: Load Data from CSV
3: Preprocess Data:
4:   Convert data types
5:   Handle missing values
6:   Normalize (Scaling) sensor values
7: Plot Sensor Data
8: for each Sensor do
9:   Filter and Clean Data
10:  Scale Data using MinMaxScaler
11:  Create Dataset  $(X, Y)$  using Time Steps
12:  Split into Train & Test Sets
13: end for
14: for each Sensor do
15:   Build LSTM Model with Layers
16:   Compile Model
17:   Train with Early Stopping
18: end for
19: Make Predictions on Test Data
20: Evaluate Model Performance:
21:   Calculate MAE, MSE, RMSE,  $R^2$  Score
22:   Plot Loss Curves & Prediction Results
23: Crossing the Threshold:
24: Define initial value  $(x_t)$ 
25: Predicted value
26: Find the minimum  $T$  where predicted value reaches  $(1 + p) \times x_t$ 
27: Save Results to File
28: End
```

Algorithm 1 demonstrates how to predict future sensor values using an LSTM model, by preprocessing and normalizing data, training the model on time-series data, and evaluating its performance. It also includes a step to predict the minimum time required for sensor values to reach a specific threshold. The algorithm provides a structured approach for accurate time-series forecasting and model evaluation.

2.7 First Evaluation

After executing the algorithm, the output revealed interesting results. The dataset defines 10 sensor IDs. The algorithm is applied separately to each sensor ID and returns results for each one. Null values are ignored for all sensors to analyze both the best- and worst-case predictions. It is important to note that when the number of sensor values is low, accuracy tends to degrade significantly. As shown below, sensor 6 exhibits poor performance.

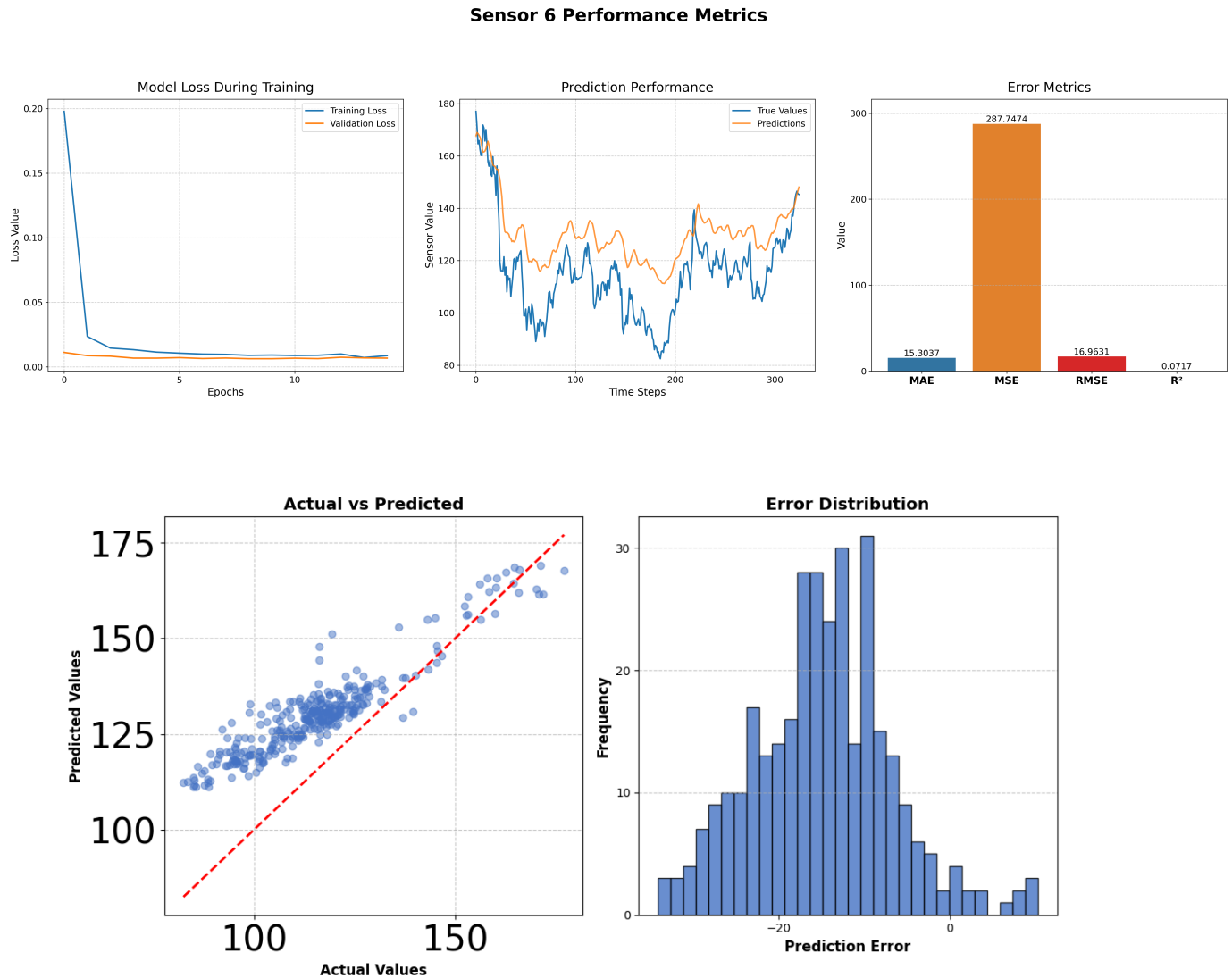


Figure 2: Error Analysis for Sensor 6

In comparison, sensor 8 performed well during training and demonstrates excellent performance. In the Actual vs. Predicted image, the X-axis represents the actual values, while the Y-axis shows the predicted values. Each blue dot corresponds to a prediction, and the red dashed line represents a perfect prediction scenario. The data points closely follow this line, indicating strong model performance, although minor overestimations and underestimations are present.

In the Error Distribution image, the X-axis represents the difference between predicted and actual values, while the Y-axis shows the frequency of each error value. Most errors are concentrated near zero, suggesting high accuracy. The distribution is centered around zero, but deviations do exist. The error range extends from approximately -10 to +10, with the highest bar exceeding 1,000 occurrences, indicating that most errors are small.

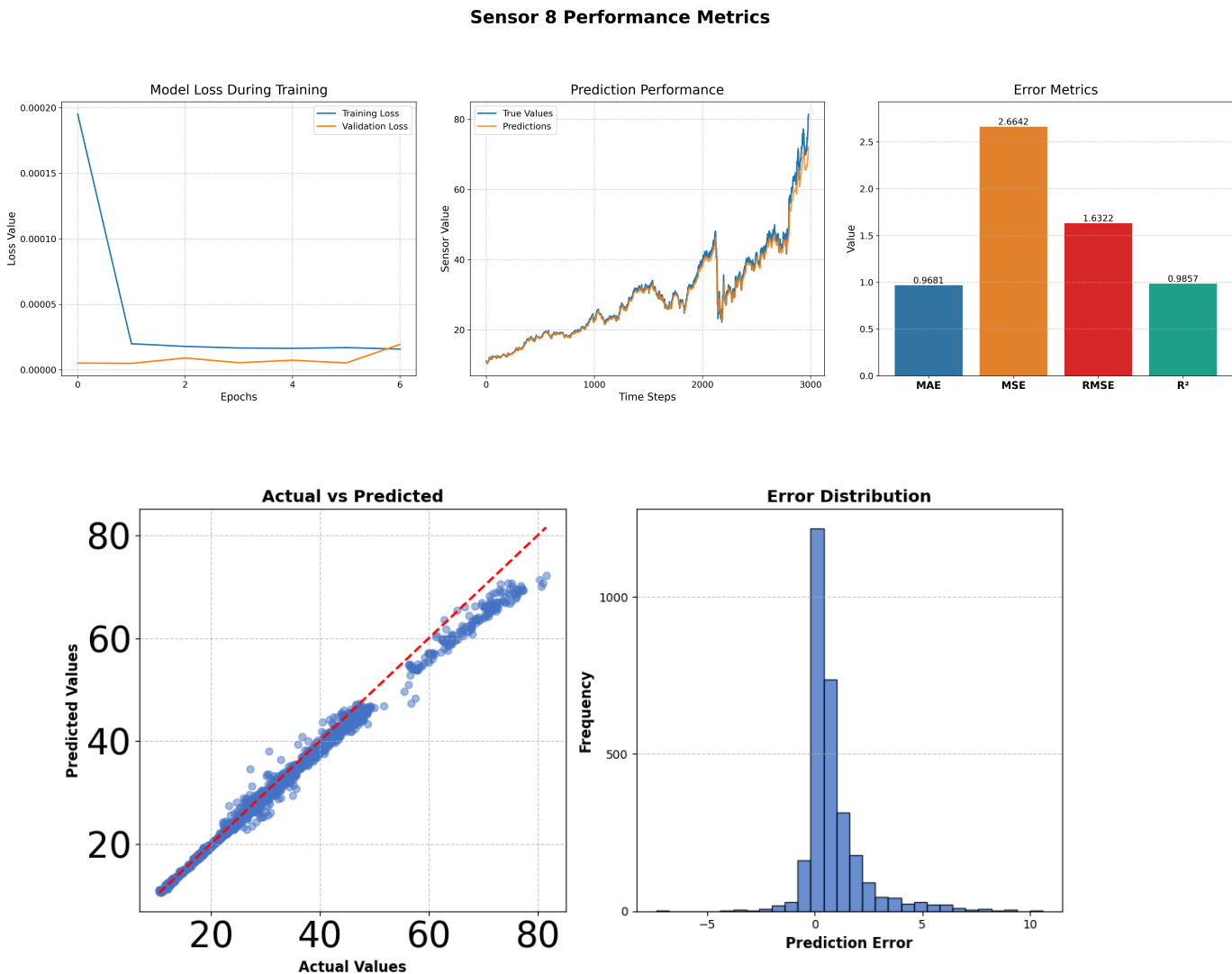


Figure 3: Error Analysis for Sensor 8

Images confirm that the LSTM model for Sensor 8 is performing well. The scatter plot and histogram indicate that errors are minimal and normally distributed. The loss curve shows

that training converged well, and the prediction performance is strong. The high R^2 score and low MAE/MSE/RMSE values suggest that the model generalizes well for real-world predictions.

2.7.1 Finding_T

As seen in Section 2.6, the algorithm takes a parameter P as input. It first takes the first element of the actual values and applies $(1 + P) \cdot x(t)$. Then, by examining the predicted values up to time t , it predicts the smallest T such that $x(t + T) \geq (1 + P) \cdot x(t)$.

Algorithm 3 Predict T When Threshold is Reached

Function $\text{predict_T}(p, x, \text{predictions})$:
 $\text{target_value} = (1 + p) \times x$
 for each (T, x_pred) **in** $\text{enumerate}(\text{predictions})$ **do**
 if $x_pred \geq \text{target_value}$ **then**
 return T
 end if
 end for
 return -1

Parameters:

predictions = predicted values list
 x = value from actual data
 $p = 0.1$ (10%)

Usage:

$T = \text{predict_T}(p, x, \text{predictions})$

Table 1: Sensor Values and Threshold Crossing Times

Sensor ID	Initial Value	Time T to Increase by 10%
0	20.413212	23
1	36.846244	53
2	11.815751	89
3	72.57335	21
4	32.681898	309
5	3.5526630	73
6	177.02000	-1
7	19.160318	73
8	11.086667	47
9	18.024734	-1

Note: $T = -1$ indicates that the sensor value never reaches the 10% increase threshold.

2.8 Second Evaluation

Here, the entire dataset is used as a single sensor ID, meaning no specific sensor ID is considered. Null values are not utilized, similar to the first evaluation. The overall number of usable values is approximately 45,000. The dataset is split into:

- Training set (80%) – Used for model learning.
- Test set (20%) – Used for performance assessment.

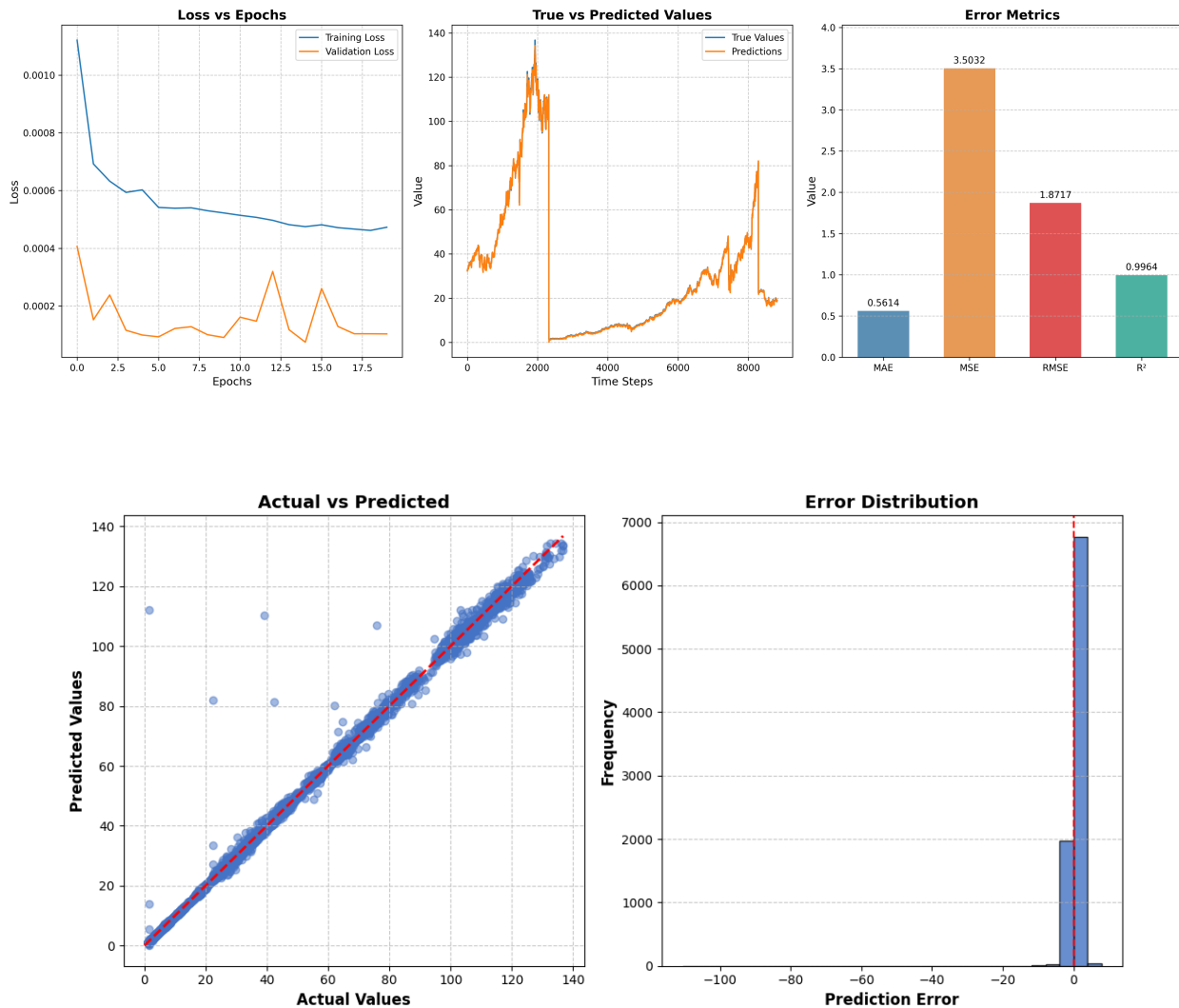


Figure 4: Error Analysis

The prediction has been done for this section as well. As you can see, it demonstrates excellent performance.

Table 2: Model Performance Metrics

Metric	Value
Time to increase by 10% (T)	54
Mean Absolute Error (MAE)	0.5614
Root Mean Squared Error (RMSE)	1.8717
Mean Squared Error (MSE)	3.5032
R ² Score	0.9964

3 Conclusion

This study presents a structured approach for forecasting sensor values using an LSTM-based time-series model. The preprocessing pipeline ensures data integrity by handling missing values, normalizing sensor readings, and restructuring data for supervised learning. The model is trained using a sliding window approach, leveraging past observations to predict future sensor values. Evaluation metrics, including MAE, MSE, RMSE, and R² score, assess the model's accuracy and reliability.

A key outcome of this work is the estimation of the minimum time required for a sensor value to reach a predefined threshold. By incorporating the future step parameter, the model generates predictions based on the average of future values, offering a smoothed estimation of sensor trends. The results demonstrate the effectiveness of the LSTM in capturing temporal dependencies and trends, making it a valuable tool for predictive maintenance.

To assess the model's performance under different conditions, various scenarios were considered. Missing values were either replaced with a constant (e.g., 1) or randomly assigned numbers, leading to variations in the obtained results. The evaluation process relied on the range of values, the number of available observations, and the presence or absence of missing data, highlighting the model's sensitivity to different data preprocessing strategies.